

融智图谱（Docker版）使用说明

背景介绍

我们知道一开始融智图谱这个项目是基于 PaddlePaddle（百度飞桨）这个深度学习框架实现的。但是由于飞桨框架起步较晚，社区规模较小，虽然发展迅速，适配了一大批国产服务器和加速卡（XPU等），但限于人力和适配难度基于 arm 架构的cpu还没有适用于大部分设备的轮子（Wheel, .whl），飞桨官方给出的方案是源码编译，但其实成功率并不高，本人尝试编译花费了40小时尚未成功编译，最终选择了将融智图谱模型重构为**基于Pytorch框架的模型**并成功在国产服务器上运行。

做出这个选择的原因是当前国外有一个由ARM官方支持的开源社区一直在对Pytorch进行arm架构适配，不过我采用的是一个个人开发者提供的Pytorch arm镜像作为基础镜像 kumatea/pytorch，原因是这个镜像是目前易得的最小可用镜像。

[KumaTea/pytorch-aarch64: PyTorch wheels \(whl\) & conda for aarch64 / ARMv8 / ARM64 \(github.com\)](https://github.com/KumaTea/pytorch-aarch64:PyTorch wheels (whl) & conda for aarch64 / ARMv8 / ARM64)

组委会提供的服务器VPS的Linux系统是 kylin serve v10，CPU是国产飞腾系列，架构是 aarch64（armv8），本次揭榜挂帅有一个评分指标是**国产化环境适配**，要求在国产CPU和操作系统、国产图数据库上，功能稳定运行。

```
[root@ncjnwr21cb7q6 ~]# cat /etc/os-release
NAME="Kylin Linux Advanced Server"
VERSION="V10 (Sword)"
ID="kylin"
VERSION_ID="V10"
PRETTY_NAME="Kylin Linux Advanced Server V10 (Sword)"
ANSI_COLOR="0;31"
```

```
[root@ncjnwr21cb7q6 ~]# cat /proc/cpuinfo | grep 'model name' | uniq
model name      : ARMV8 CPU
```

我们的项目目前提供了两个主要的后端api，其余后续操作均可采用 JSP 实现，目前两个后端api都是基于Pytorch实现，并已上传至 Docker Hub 公开仓库。

[onefly/torch-backend-api-general | Docker Hub](https://hub.docker.com/r/onefly/torch-backend-api-general)

[onefly/uie-backend-api-general | Docker Hub](https://hub.docker.com/r/onefly/uie-backend-api-general)

构建镜像

为了方便使用国产服务器的后来贡献者，我将我采用的Docker镜像构建模板放在下方，方便大家根据自己的项目要求灵活构建。

```
# 使用 PyTorch aarch64镜像作为基础
FROM kumatea/pytorch

## 其他 Docker 指令，例如设置工作目录、复制文件等。
# 设置工作目录
WORKDIR /app

# 将 torch 文件夹的内容复制到容器的 /app 目录中
```

```
COPY ./torch /app

# 安装项目所需的依赖
RUN pip install --no-cache-dir -r requirements.txt

# 设置 Flask 环境变量
ENV FLASK_APP=torch-backend-api.py
ENV FLASK_RUN_HOST=0.0.0.0

# 暴露端口 999
EXPOSE 999

# 运行 Flask 应用
CMD ["python", "/app/torch-backend-api.py"]
```

所需命令：

需要进入Dockerfile所在目录

```
cd ./docker
```

然后开始构建镜像，如果是Windows上需要先启动客户端软件：

```
docker buildx build --platform linux/arm64 -t onefly/uie-backend-api:arm64 . -f
Dockerfile --load
```

ps:

`buildx` 是一个跨平台构建试验性功能，可以构建与自身芯片架构和系统不同的镜像版本，`--platform linux/arm64`表示构建armv8版本的镜像

`-t` 是tag标记的意思，即在build的时候就标记image的名字和版本

`-f` 是指定DockerFile文件路径和文件名的参数

与之类似的是构建文本补全的api的命令：

```
docker buildx build --platform linux/arm64 -t onefly/torch-backend-api:arm64 . --
push -f Dockerfile2
```

想要上传镜像的Docker Hub需要以下命令：（先在Docker Hub网页端新建一个同名Repo）

```
docker login
docker tag onefly/uie-backend-api:arm64 onefly/uie-backend-api:arm64
docker push onefly/uie-backend-api:arm64
```

运行容器

运行容器的第一步是拉取镜像：

```
docker pull onefly/uie-backend-api:latest
docker pull onefly/torch-backend-api:arm64
```

然后首先在有终端的情况下运行（便于debug）：

```
docker run -p 888:888 onefly/uie-backend-api:arm64
```

通过 Postman 等api调试工具验证无误后即可在后台运行,添加参数 `-d`：

```
docker run -p 888:888 onefly/uie-backend-api:latest
```

```
docker run -p 999:999 onefly/torch-backend-api:arm64
```

大功告成！

其他常用的docker命令：

查看当前的镜像列表：

```
docker images
```

强制删除无用镜像，失败镜像(image id)：

```
docker rmi -f xxxxxxxx
```

查看docker容器运行情况：

```
docker ps
```

停止正在运行的容器（container id）：

```
docker stop xxxxxxxx
```

服务器重启后启动docker服务：

```
sudo systemctl start docker
```

立即重启服务器：

```
sudo reboot
```

模型重构

模型重构指的是我们将基于Paddle框架的UIE模型重新手动编写为不依赖飞桨框架的Pytorch版本，使用此版本可以不再需要安装数个GB的飞桨框架和PaddleNLP这个库，真正实现独立自主完成本项目。本Pytorch版本基于UIE模型在其论文中提到的思路复现得到，主要区别在于飞桨版本是以指针标注方式（双指针解码）构建的抽取式版本，而Pytorch版本是生成式。

虽然寥寥几字和几个处理就能实现转换，但我们在重构和实现过程中经历了千辛万苦。最为核心的工作由两部分：

1. 将UIE的微调模型转换成支持Pytorch的模型文件。

2. 将Pytorch模型由动态图推理模型转换成静态图模型，推理引擎由 Pytorch 切换为 onnx，如此以支持在arm架构的国产服务器上运行。

以上两个步骤在项目目录uie-pytorch均有一键shell命令可以实现：`convert.sh`、`export.sh`。